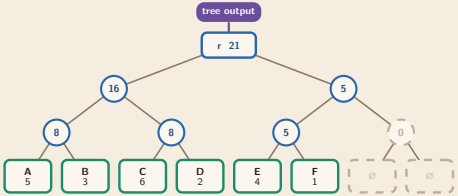
LayerTree

One UTXO, an off-chain Merkle-sum tree, and a covenant that lets anyone leave in one transaction

Olaoluwa Osuntokun `roasbeef`



In five lines

LayerTree in five lines

1. One output commits to a Merkle-sum tree of balances; the tree is data, not transactions.
2. Every transition proves an old node and folds a new one up the same path; the sums make theft structurally impossible.
3. Anyone leaves in **one transaction** with their own signature: 1,020 vB at 65,536 members from proposed opcodes (FULL); about 541 vB, plus 10 vB per doubling, with a hypothetical Merkle-sum opcode.
4. Payments happen off-chain: channel leaves make it a one-UTXO Lightning hub, vouchers and pending pairs give zero-byte receives.
5. The cost is one queue for everyone, and opcodes that aren't active yet. Four tiers show exactly how few are needed.

`layertree.lightning.wiki` `spec/layertree.md` `paper/main.pdf`

The problem

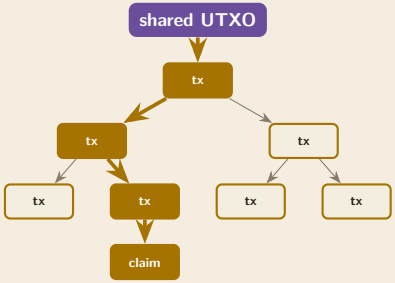
Scaling is about ownership, not payments

- Every distinct owner wants their own unspent output, and outputs are the scarce resource.
- So share one output between many owners. The hard part is not sharing, it's **leaving**.
- Channel factories, CoinPool, Ark, timeout trees, CTV and CCV trees all answer the same way: the shared output is the root of a **tree of transactions**.
- To leave alone you publish the path from the root to your leaf: $\log_2 N$ transactions, each with a fee and a confirmation, often against a deadline.

You pay for the sharing at exactly the moment you most need it to be cheap: when the operator is gone, fees are high and everyone else is leaving too.

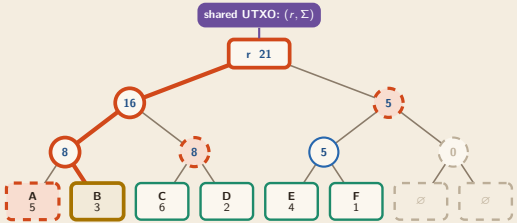
Two shapes of shared output

A transaction tree (Ark, CTV, timeout trees)



leave alone: $\log_2 N + 1$ transactions,
each confirmed in order, before the round expires

LayerTree



the tree is off-chain data; leave alone: **one**
transaction that proves your leaf to the output itself

The idea: the tree is data, and the output checks the proof

To leave, present your leaf and its Merkle proof **to the output**. The spending script:

1. recomputes the root from your leaf and proof,
2. checks it matches the root the output commits to,
3. recomputes the root with your leaf removed,
4. forces the transaction to recreate the shared output under that root and pay you.

One transaction. No pre-signed tree. Nobody else's signature. No timelock.

What it costs

About **32 vB per level** with proposed opcodes (FULL tier), **10 vB per doubling** with a hypothetical native Merkle-sum opcode.

At 65,536 members: **1,020 vB** (FULL), **908 vB** (CCV), about **541 vB** (CCV + the hypothetical opcode).

Constant in N up to the depth: it does not grow with the number of members.

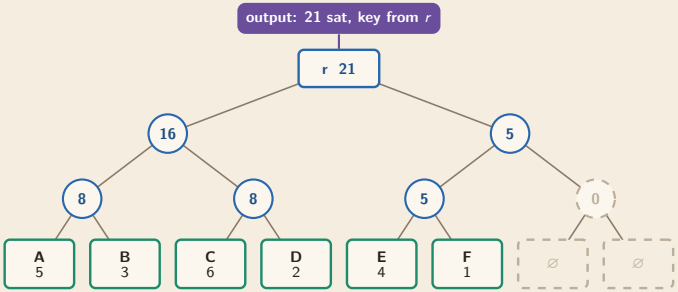
The construction

A leaf is a 116-byte record



- `exit_key` $\neq$ `owner_key`: a hot key authorises, a cold key (or a channel's funding key) receives.
- `evict_after`: a *lease*. Before it, nobody can remove you. After it, the evictor can, and only by paying you in full.
- `seq`: bumped by every carve, which is what makes vouchers single-use. `SEQ_FINAL` marks a leaf whose amount is not the owner's alone (channel leaves).
- Node of a leaf: $(H_{\text{leaf}}(\text{leaf}),\ \text{amount})$.

A Merkle-*sum* tree



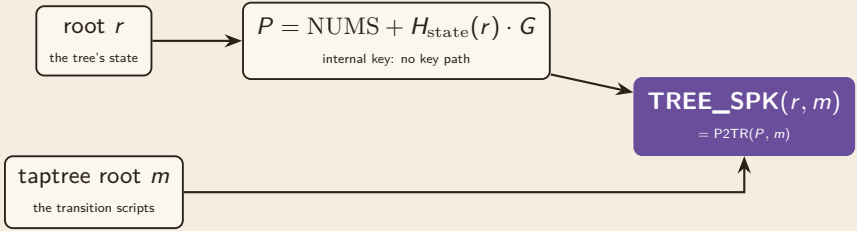
Every node is a pair: a hash and an amount.

$$\left(H_{\text{node}}(l_h \| \text{enc}(l_s) \| r_h \| \text{enc}(r_s)),\ l_s + r_s\right)$$

The root commits to *who* is in the tree and *how much* they add up to, and the output's value must equal the root's sum.

A script sees one leaf and one path, but because sums travel with hashes, verifying the path verifies the leaf's **share of the total**. To steal you'd need a different tree with the same root.

State lives in the output's address



- Change one balance and the address changes. The output *is* the state machine.
- Scripts get the claimed r and m in the witness, recompute the output key, and compare it with the coin being spent (and the one being created). Lie about the root and the keys don't match.
- m binds the rule set too: you can't recreate the output under rules of your own choosing.
- Under CCV (BIP-443) the same binding is the data tweak `sha256(NUMS || r)`, checked natively by `OP_CHECKCONTRACTVERIFY`.

Every transition is the same machine

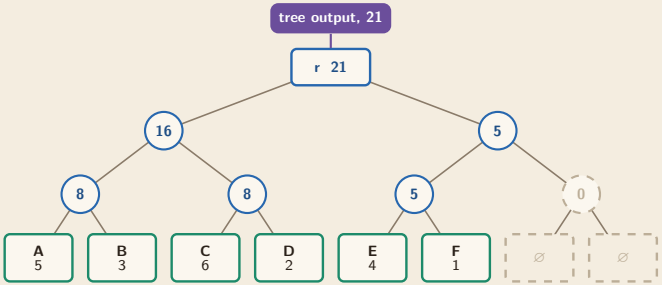
A **step**: prove an old node against the current root, then fold a new node up the *same* siblings to get the next root.

- exit: leaf $\rightarrow$ empty
- join: empty $\rightarrow$ leaf or subtree
- update: leaf $\rightarrow$ leaf
- carve: two steps, shrink one leaf, fill an empty slot

Tapscript has no loops, so every batch size is a different script: a different tapleaf in the taptree, chosen at creation.

Transition	Who	Does
EXIT	owner	leaf $\rightarrow \varnothing$, pay exit key
JOIN	operator	$\varnothing \rightarrow$ leaf / cohort
UPDATE	owner + op	rewrite a leaf
EVICT	evictor, after lease	exit, paid in full
CLAIM	voucher holder	carve a new leaf
SPLIT	owner	carve, signed directly
REWRITE$_h$	changed owners + op	a cohort at once
DETACH$_h$	operator	cohort to own output
RECLAIM	op, after deadline	pending leaf back

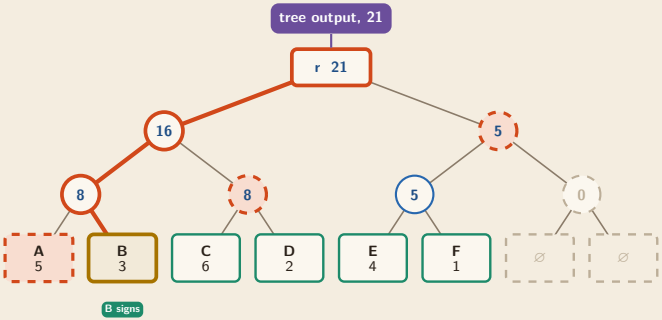
The covenant makes three checks



- **The input commits to the old root.** Fold B's leaf with its three siblings; derive the key; compare.
- **The rule holds.** B signed; B's payout is exactly 3 to B's exit key.
- **The output commits to the new root.** Fold ∅ up the same siblings; output 0 must be TREE_SPK(r') with 18.

The chain holds a commitment and an amount. Everything else lives off-chain with the members.

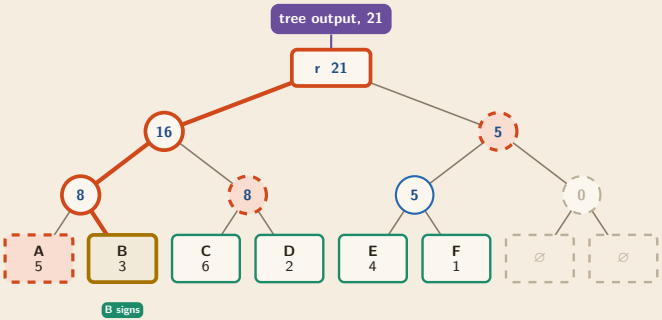
The covenant makes three checks



- ✓ **The input commits to the old root.** Fold B's leaf with its three siblings; derive the key; compare.
- ○ **The rule holds.** B signed; B's payout is exactly 3 to B's exit key.
- ○ **The output commits to the new root.** Fold ∅ up the same siblings; output 0 must be TREE_SPK(r') with 18.

Witness: B's leaf, a sibling per level, r, m, B's signature.

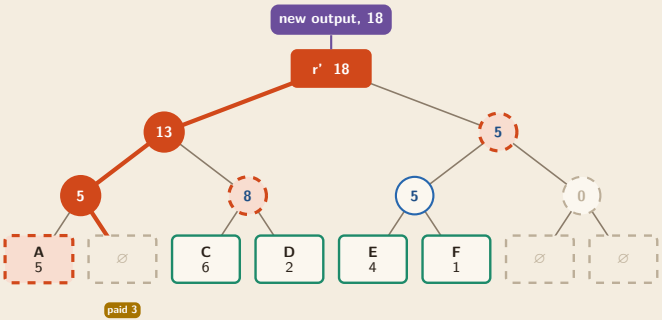
The covenant makes three checks



- ✓ **The input commits to the old root.** Fold B's leaf with its three siblings; derive the key; compare.
- ✓ **The rule holds.** B signed; B's payout is exactly 3 to B's exit key.
- ○ **The output commits to the new root.** Fold ∅ up the same siblings; output 0 must be TREE_SPK(r') with 18.

The rule is the only part that differs between transitions.

The covenant makes three checks

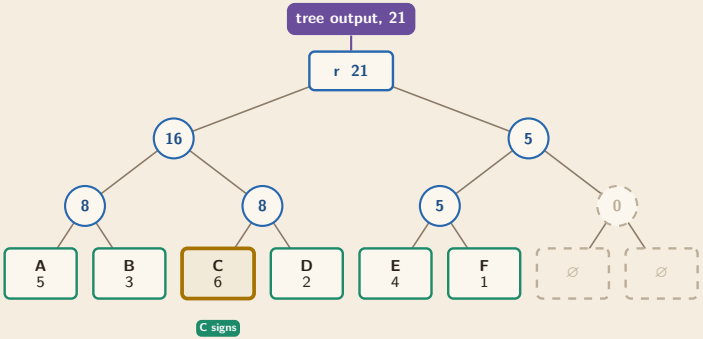


- ✓ **The input commits to the old root.** Fold B's leaf with its three siblings; derive the key; compare.
- ✓ **The rule holds.** B signed; B's payout is exactly 3 to B's exit key.
- ✓ **The output commits to the new root.** Fold ∅ up the same siblings; output 0 must be TREE_SPK(r') with 18.

Same siblings, folded twice. That's the whole cost, and it depends on depth, not membership.

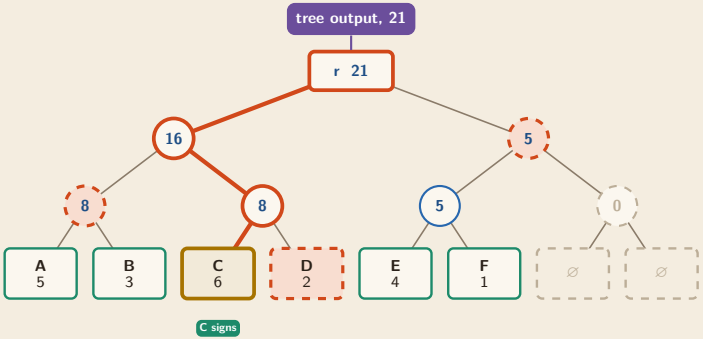
The transitions, step by step

EXIT: leaving needs only your own signature



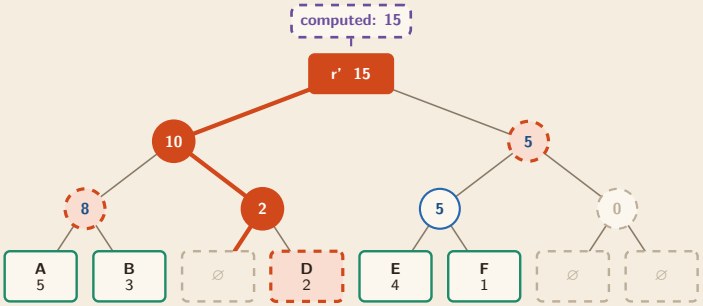
C decides to leave. No permission, no counterparty, no pre-signed transaction: C builds a transaction spending the tree output and signs it with the owner key.

EXIT: leaving needs only your own signature



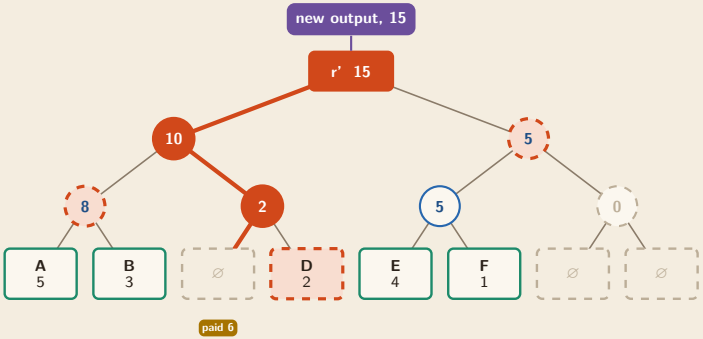
The witness carries C's leaf and three siblings: D, the node over A and B, the node over the right half. Folding gives $8 \to 16 \to 21$: the committed root.

EXIT: leaving needs only your own signature



Fold again with $\varnothing$ in C's place, same siblings: $2 \to 10 \to 15$.

EXIT: leaving needs only your own signature

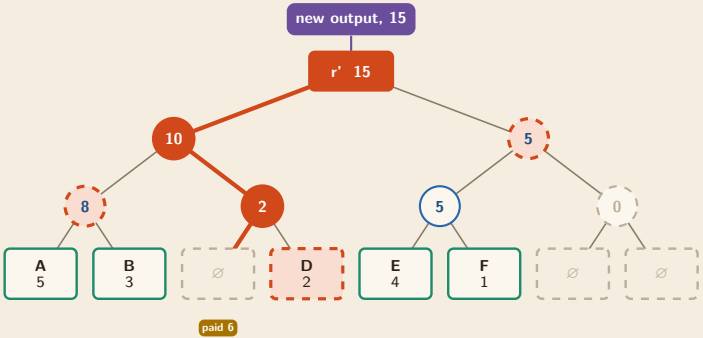


Output 0 recreates the tree with exactly 15; output 1 pays C's exit key 6.

Depth 16: 1,020 vB (FULL), 908 (CCV).

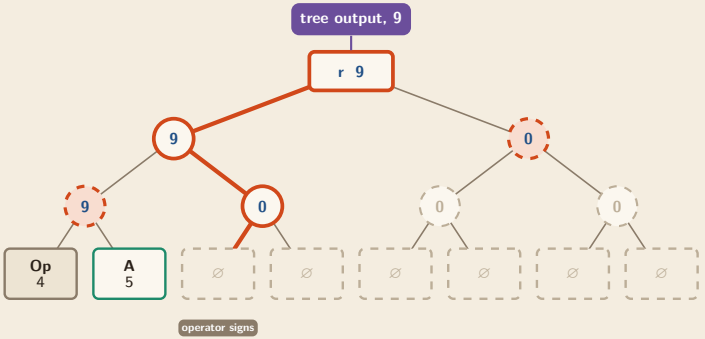
Exact-payout trees (**EXIT**$^{=}$) pay fees from a separate input, so nobody can shave a payout.

EXIT: leaving needs only your own signature



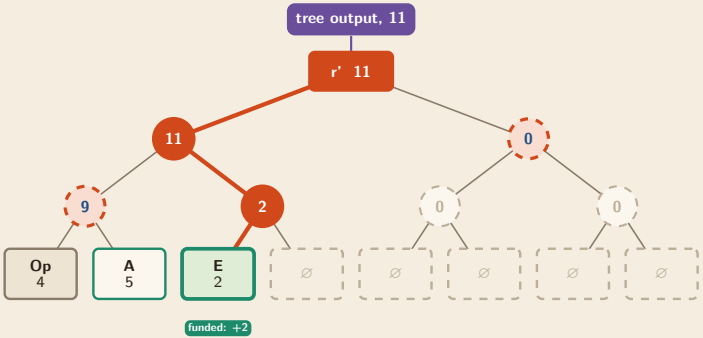
EXIT$^{t}_{K}$: the owner signs once, off-chain, over $H_{\text{exit}}(\text{tree_id} \,\|\, \text{body(leaf)})$. A tower builds and rebuilds the exit with the owner offline. It survives top-ups, and pays only the owner's exit key: a bearer right to *when*, never *where*.

JOIN: one member, or a whole cohort for one price



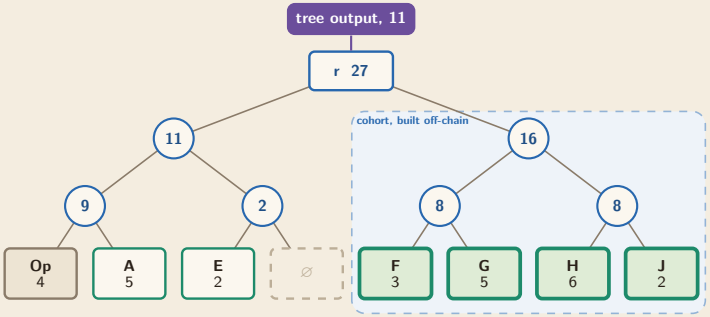
Only the operator adds leaves. A single join proves that a slot holds the empty node, with the same kind of proof an exit uses.

JOIN: one member, or a whole cohort for one price



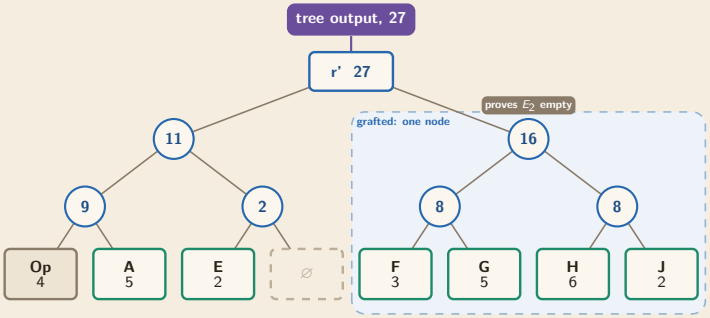
E's leaf folds up the same path. The output must grow by exactly 2, brought in by another input. **About 937 vB at depth 16.**

JOIN: one member, or a whole cohort for one price



For a cohort the operator builds a subtree off-chain and sends each joiner their leaf and path. **Each joiner checks their own path before paying.**

JOIN: one member, or a whole cohort for one price

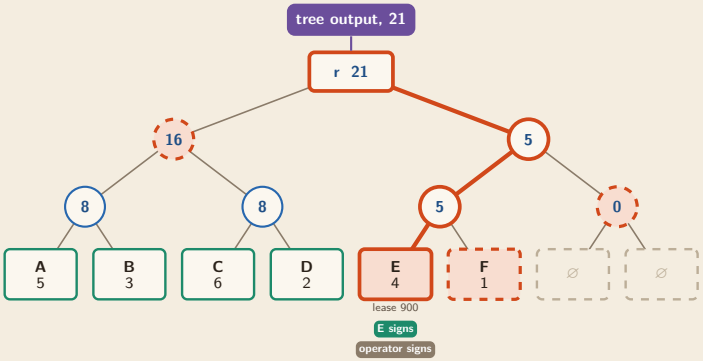


The graft proves one empty node of height 2 and replaces it with the cohort's node. The covenant never looks inside.

256 members: 679 vB. 1,024: 614 vB. Under 1 vB per member, against 43 vB for an output in a batch.

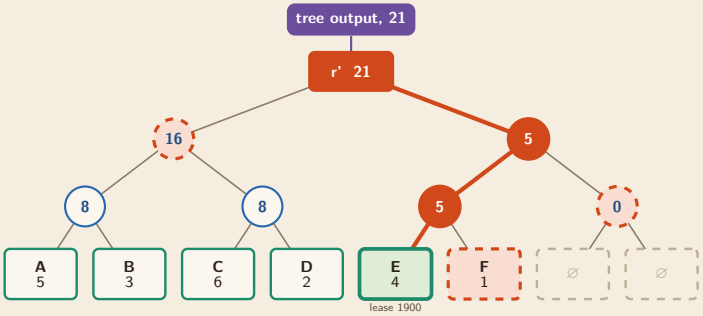
A bad graft freezes value, it can't create or steal any: the sums still balance.

UPDATE, and the top-up that needs no owner



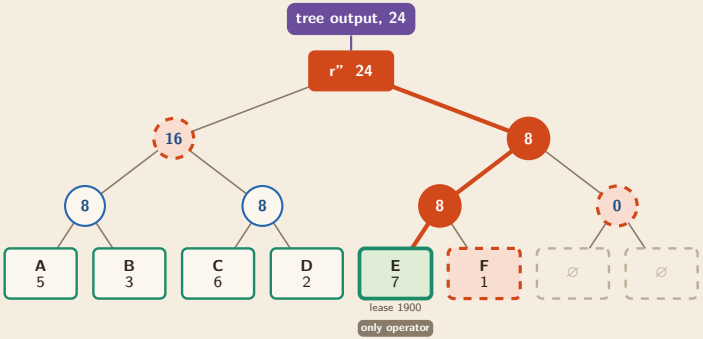
UPDATE rewrites a leaf in place: resize, renew the lease, rotate keys. It needs the owner *and* the operator.

UPDATE, and the top-up that needs no owner



Here E renews its lease. One step, about 1,030 vB alone, which is why payments must not do this one at a time.

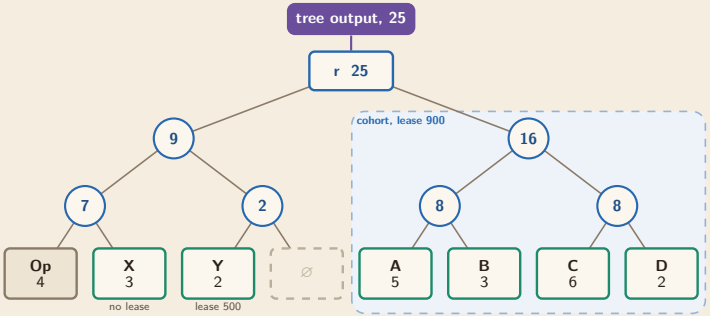
UPDATE, and the top-up that needs no owner



An **amount-only increase** needs no owner signature: nobody can use it to rotate a key or shorten a lease. The operator tops E up by 3 from its own funds.

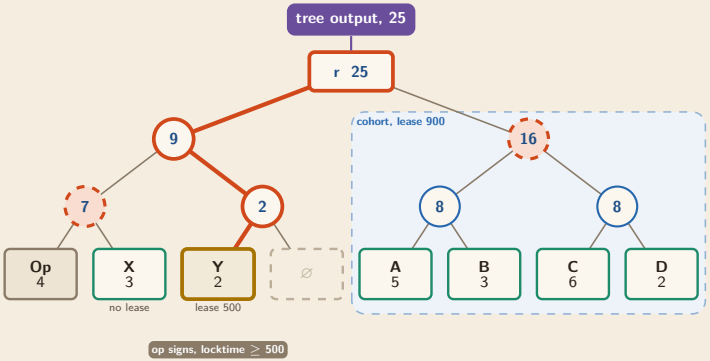
Except on a `SEQ_FINAL` leaf: a channel leaf's amount is the capacity its signed states commit to, so changing it needs the owner.

Leases, eviction and the cohort sweep



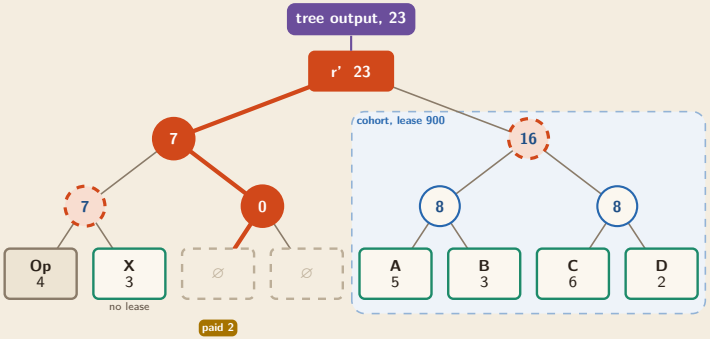
Every leaf names an evictor and a lease height. X's evictor is NUMS: X can never be evicted.

Leases, eviction and the cohort sweep



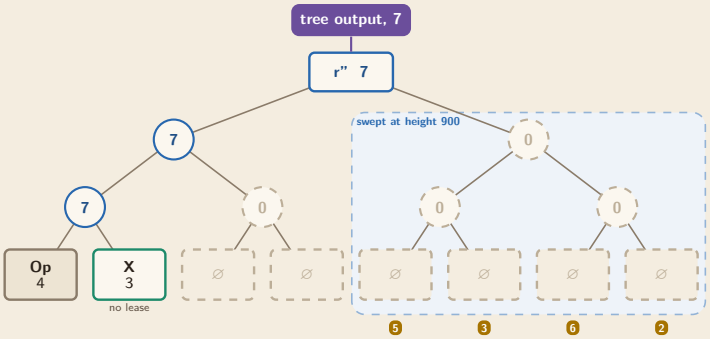
An eviction is an exit signed by the evictor, in a transaction whose `nLockTime` is at least the lease.

Leases, eviction and the cohort sweep



The payout must equal Y’s amount exactly. The operator can remove Y but cannot take a satoshi of it.

Leases, eviction and the cohort sweep

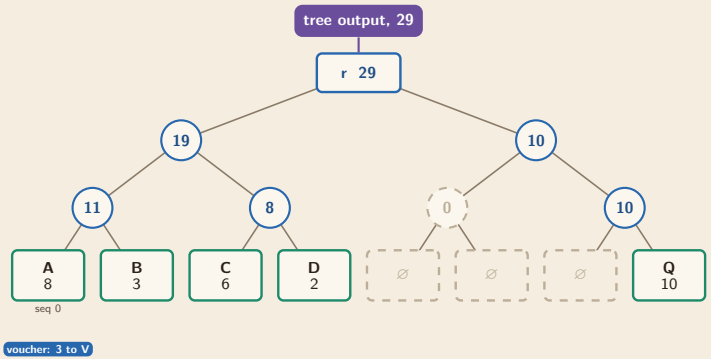


EVICT_SUBTREE$_h$ publishes the cohort's slots once, checks every lease, pays every leaf, and proves one path.

13 leaves: 203 vB each. 51 leaves: 158 vB each.

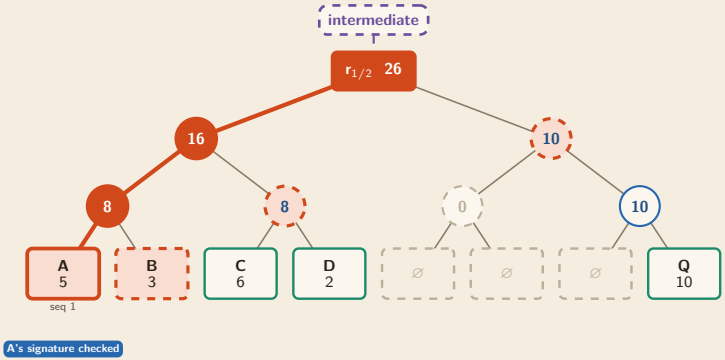
Cheap eviction cuts both ways: it reclaims dead weight and it makes griefing cheap. Members who want a hard guarantee hold a long lease.

Vouchers: CLAIM carves a new leaf out of yours



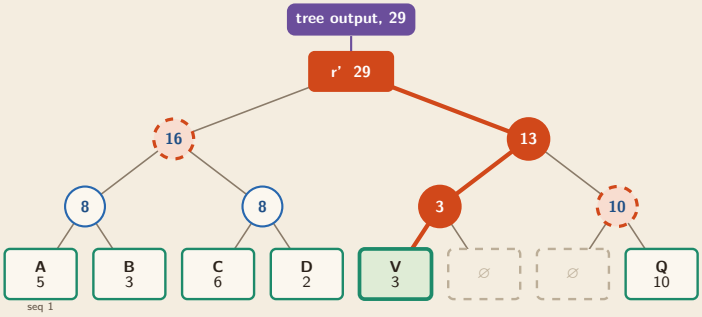
A signs a message naming its own leaf (without the amount), `tree_id`, and a new leaf giving 3 to V. Nothing happens on-chain.

Vouchers: CLAIM carves a new leaf out of yours



V's **CLAIM** proves A's leaf and checks A's signature with `OP_CHECKSIGFROMSTACK`. A's leaf shrinks by 3 and its seq goes up.

Vouchers: CLAIM carves a new leaf out of yours



A second proof, under the intermediate root, shows slot 4 is empty, and V's leaf goes there. Total unchanged.

Single use for free: the message binds A's seq, so it never verifies again.

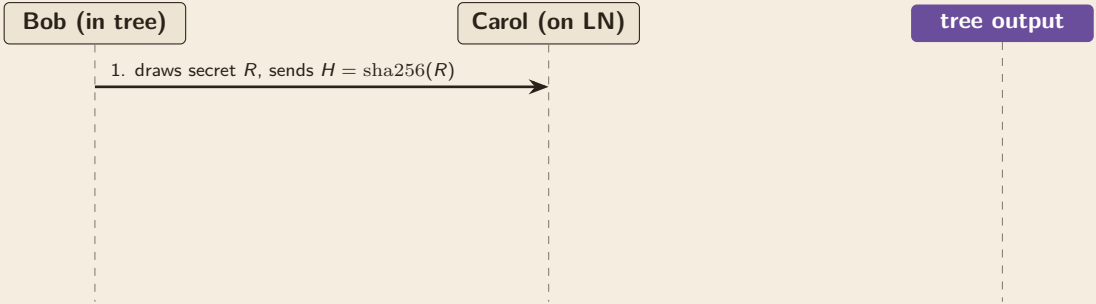
The covenant stops over-redemption, not over-issuance: an unredeemed voucher is credit.

The voucher family

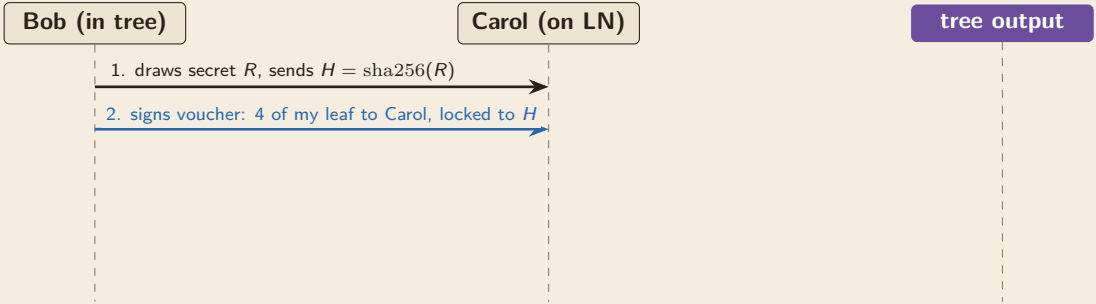
Form	What it adds	What it's for
CLAIM	owner's signature over the carve	pay someone not yet in the tree
SPLIT	signature over the transaction instead	the same carve without CSFS: exists in MIN
CLAIM^{h}	a 32-byte preimage hashed *into* the signed message	an HTLC: atomic swaps against Lightning
CLAIM^{x}, ^{hx}	a signature by the source's *exit* key too	a credit leaf the operator can't drain
CLAIM^{m}, ^{hm}	target is the sibling, raised to an absolute amount	the atomic receive; tower-redeemable

- The hash lock costs 8 bytes and no opcode: a wrong preimage just produces an invalid signature.
- A voucher can have no on-chain deadline: script can say "not before", never "not after". Deadlines live in the other leg of a swap, or in the source leaf itself (the pending leaf, later).
- Vouchers bind `tree_id`, unique per lineage, so they can't be replayed against a sibling tree with the same rules.

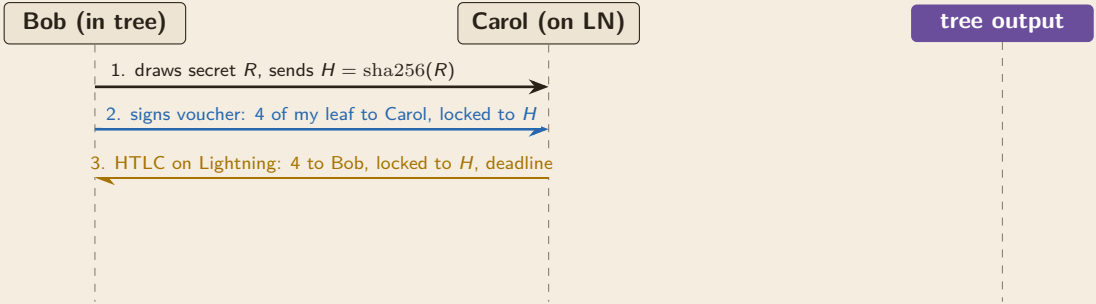
Swaps: a voucher and an HTLC on one secret



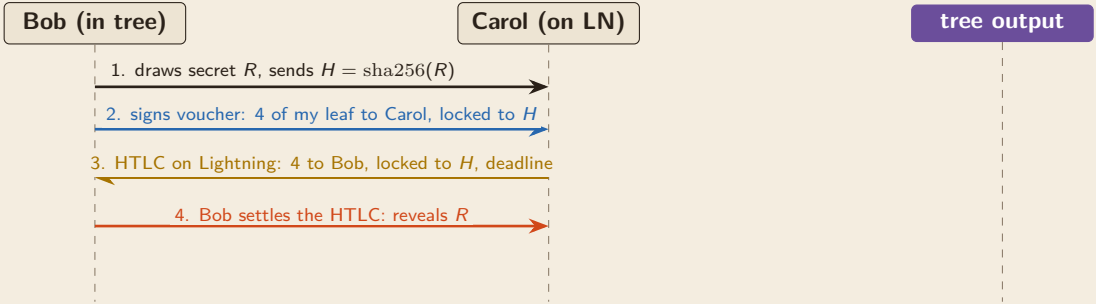
Swaps: a voucher and an HTLC on one secret



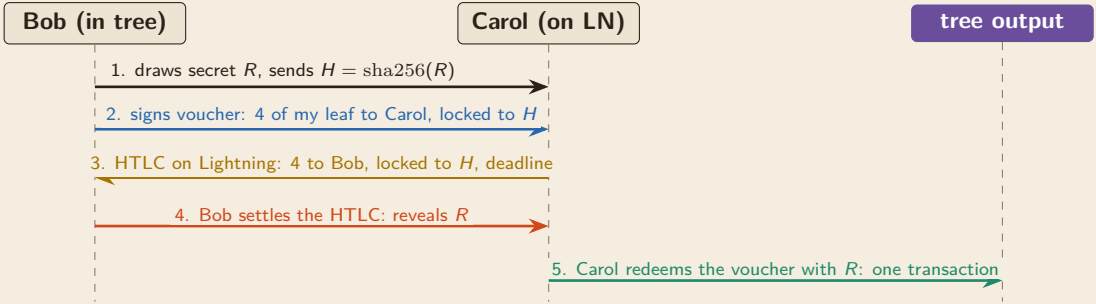
Swaps: a voucher and an HTLC on one secret



Swaps: a voucher and an HTLC on one secret

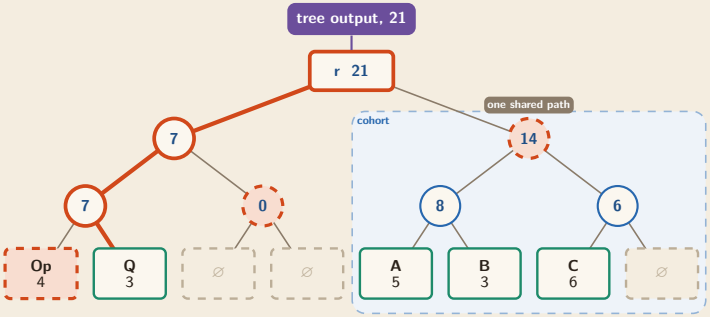


Swaps: a voucher and an HTLC on one secret



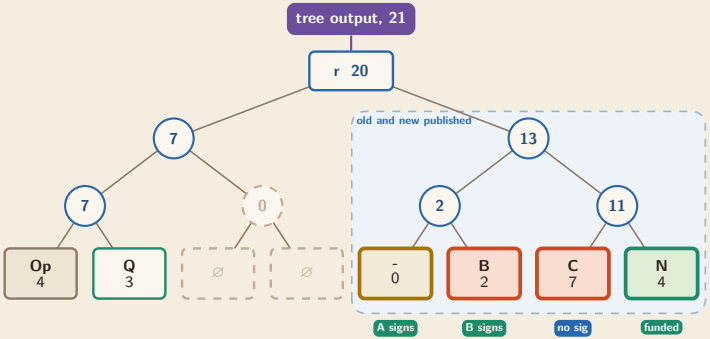
- The HTLC has a deadline and the voucher doesn't, which fixes the order: the secret comes from the party the HTLC pays, and the deadline-free leg resolves *second*.
- Done right, the Lightning leg settles off-chain and the whole swap is one transaction. Buy a leaf with Lightning sats while holding no on-chain bitcoin at all.

REWRITE$_h$: a cohort in one transaction



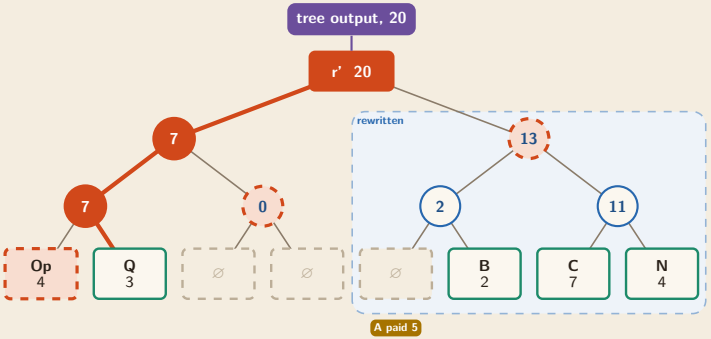
A, B and C were grafted together, so above their subtree they share one path.

REWRITE$_h$: a cohort in one transaction



In one rewrite: A leaves, B pays C 1, N joins. The witness has every slot twice, before and after. A and B sign because their leaves change. C's change is amount-only, so C needn't sign. The operator signs once.

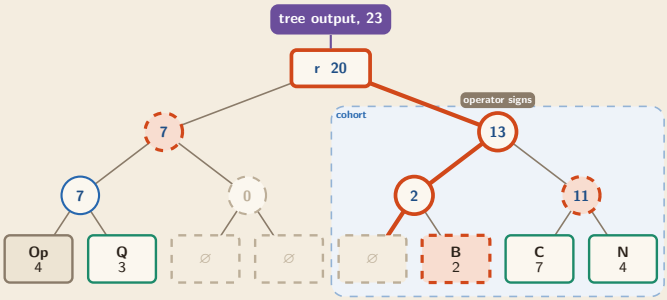
REWRITE$_h$: a cohort in one transaction



The script rebuilds the old subtree, checks it against the root, rebuilds the new one and folds it up the same path. A is paid exactly 5; N's 4 comes from an input.

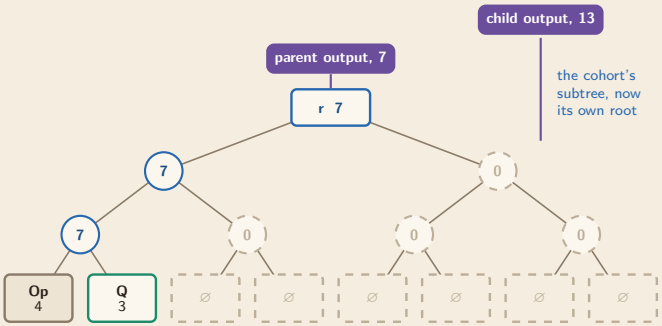
64 positions: 151 vB per changed leaf (vs 1,030 per UPDATE), 165 per exit, 95 per leaf left alone (a one-byte "unchanged" marker).

DETACH$_h$: a cohort gets its own output



The operator moves an aligned subtree into a tree output of its own, under the same option set and `tree_id`.

DETACH$_h$: a cohort gets its own output



The operator moves an aligned subtree into a tree output of its own, under the same option set and `tree_id`.

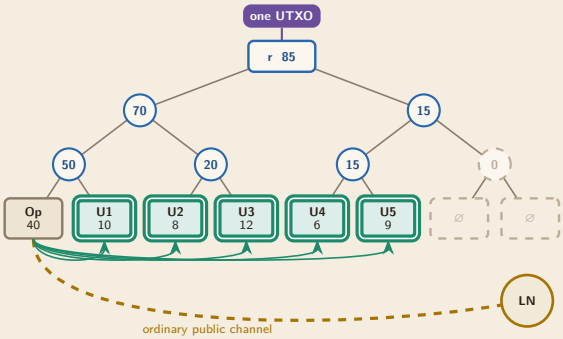
The cohort's node becomes the root of a depth-h tree. Members' proofs still work: they just stop $D - h$ levels early.

- later exits pay h levels, not D
- the cohort gets its own mempool chain
- about 780 vB (621 under CCV)

It's the structural answer to contention: shard on demand.

Lightning

Channel leaves: a Lightning hub with one UTXO



A leaf's `exit_key` is the funding output key of a 2-party channel with the operator (MuSig2, LN-Symmetry update machine, Appendix C).

- The funding output doesn't exist yet. It materialises at whatever transition removes the leaf.
- Payments between channel leaves are ordinary HTLC forwards through the operator. **Nothing touches the tree**: no spend, no proof, no fee.
- Same hash and timelock arithmetic as the public network, so it routes like any LN node.

The price of a channel leaf is patience

- A force close must first get an **EXIT** confirmed, and that exit competes with every other transition for the single output.
- So HTLC deadlines need about **220 blocks** of `cltv_expiry_delta` headroom, against 40 to 80 on the public network. The close (exit, update, settlement) takes about **146 blocks**.
- The leaf is `SEQ_FINAL`: not voucher-claimable, and no amount-only top-up without the owner, because its amount is the capacity every signed state commits to.
- The lease is the operator's force-close right: `evict_after` is when the operator may put the channel on-chain, and HTLCs must expire before it plus the budget.
- `chan_id` binds `tree_id`, so channel states can't be replayed into a sibling tree.
- A tower holding an $\mathbf{EXIT}_K^t$ authorisation and the latest state can *start* the close, not only answer a stale one.

Spec: Appendix C (`spec/channel-leaves.md`), C.4.4 deadlines, C.6 rules that bind the tree.

Receiving without a channel: the credit leaf

A user with no on-chain funds wants to receive over Lightning.

- The operator funds a **credit leaf** for them.
- Each receive: the payer's HTLC reaches the operator, the operator settles it, then signs one cumulative hash-locked voucher for the running total.
- Zero on-chain bytes per payment, no minimum, never expires; the recipient can redeem the latest voucher alone.
- **CLAIM**^{x} + **UPDATE**^{x} + exact payouts: the credit leaf can only grow or pay its recipient.

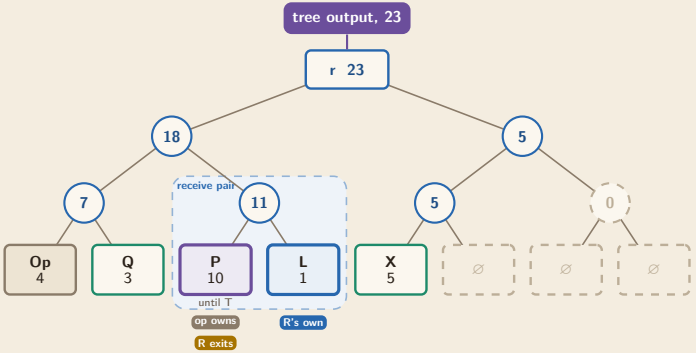
The gap: it settles first

Between settling the HTLC and signing the voucher, the operator holds the payment and the recipient holds nothing enforceable.

Exposure is one payment, but it's trust.

Closing it needs the recipient to hold something *before* releasing the preimage, and the operator to get its money back if the preimage never comes. That's a deadline, which a voucher can't carry.

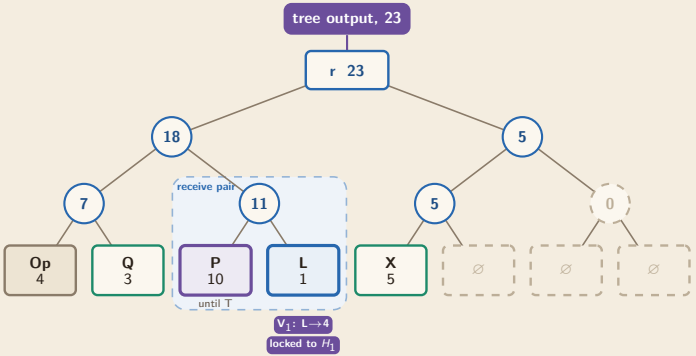
Pending pairs: an HTLC that lives in the tree



P, pending: owned by the operator, exits to the recipient, evictor `RECLAIM_KEY`, deadline T. Funded with the window's expected receipts.

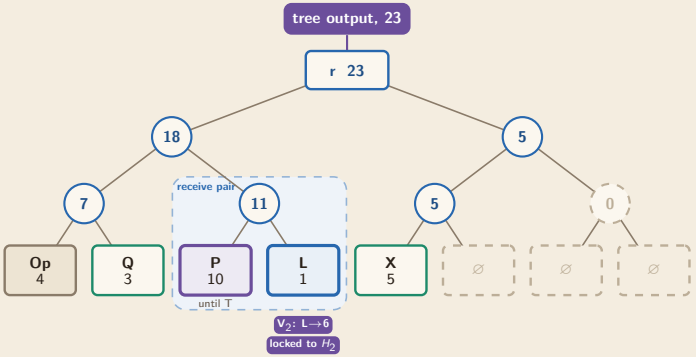
L, landing: the recipient's own leaf, long lease.

Pending pairs: an HTLC that lives in the tree



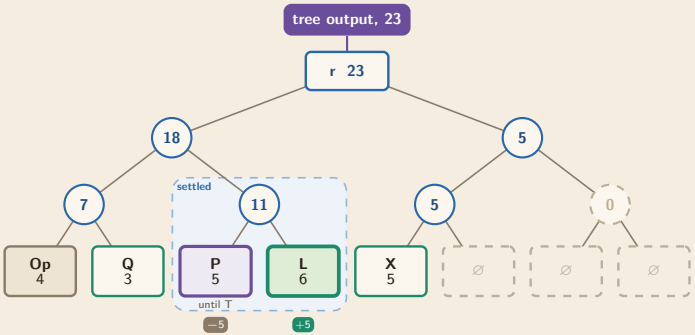
A payment arrives (hash H_1). Before releasing the preimage, the recipient gets a hash-locked **merge voucher**: "raise P's sibling to 4". Then it reveals the preimage, and the operator settles. **Atomic**: neither side can walk.

Pending pairs: an HTLC that lives in the tree



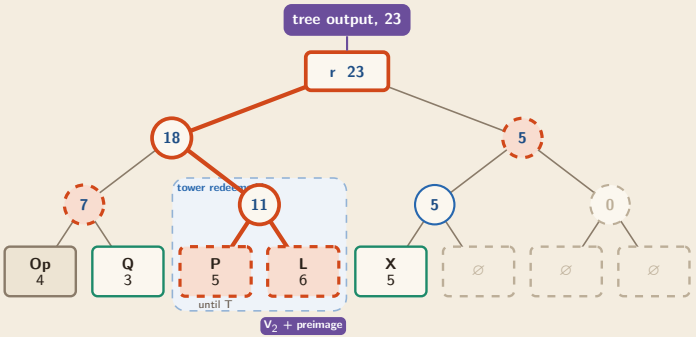
Another payment: a new voucher to an *absolute* target, $L_0 + \Sigma a_i = 6$. Replay is harmless: a target can only be reached once. One voucher per window, however many payments.

Pending pairs: an HTLC that lives in the tree



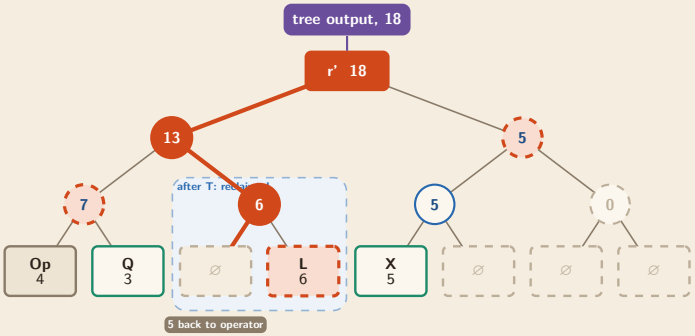
R3, the normal path. Before $T - \Delta$ the operator's reclaim-form rewrite moves the settled total from P to L, with *only the operator's signature*. Batched across many pairs.
Non-interactive: the recipient can be asleep.

Pending pairs: an HTLC that lives in the tree



R2, if the operator doesn't. A tower redeems the latest voucher with the preimage (**CLAIM**^{hm}: about 1,136 vB in FULL, 626 with the hypothetical opcode). It can only move value *into* a leaf the recipient alone exits, so the tower holds no key.

Pending pairs: an HTLC that lives in the tree



R1, after T. The operator reclaims what's left of P with a `nLockTime` $\geq T$ (**RECLAIM**$_1$), and must hold the inbound HTLCs until that confirms. Then it rolls a fresh pair.

R1, R2, R3: three rules, no new opcode

	Rule	Why it's shaped this way
R1	A leaf whose evictor is `RECLAIM_KEY` can be removed after its deadline with no payout	the HTLC's timeout branch; a key, not a flag bit, because MID can't isolate a bit
R2	**CLAIM**^{m}: raise the source's *sibling* to an absolute target; target owner = target exit = source exit key	a target the operator could exit or graft could void or drain the voucher; Lean found the exit-key equality was needed
R3	In the reclaim-form rewrite, an even reclaimable leaf may decrease amount-only if its odd sibling rises by exactly the same, with only `sig_op`	batch settlement with nobody online; any other change to a reclaimable leaf takes the R1 branch

Per pair, per window (CCV + opcode)

no R3: **CLAIM**^{m} + **RECLAIM**$_1$	1,105 vB
R3 batch, 16 positions	624 vB
R3 batch, 64 positions	547 vB

Per payment: **zero**. The cost moves to the window, so a merchant with a thousand receives a day pays what a wallet with one does. Built end to end in FULL, MID and CCV; MIN can't (no CSFS). These figures assume a proof that can start above height 0; without that, about 480 vB more per merge or rewrite.

Towers: they choose when, never where

Job	Holds	Deadline it serves
keep the path current	leaf, path, the chain	none; everything builds on it
start an exit	an $\mathbf{EXIT}_K^t$ authorisation, a fee UTXO	the owner's own, or a lease
answer a stale channel state	latest update + settlement	`csv_delay`
redeem a pending-pair voucher	merge voucher, preimage	the pending leaf's T
bump fees, outbid conflicts	a funded UTXO	the contention bound $\Delta(q)$

- Every delegable instrument pays a fixed output that belongs to the member. Misuse is bounded to timing: a forced exit of your own funds, paid in full.
- Light towers can't watch a fixed scriptPubKey (it changes every transition); they follow the outpoint chain, or ask the operator and check against the confirmed root.
- Not designed yet: a tower protocol (registration, payment, hand-off messages).

Opcodes and tiers

No new opcodes: four tiers from existing proposals

Tier	Needs	Carries	Exit, 2^{16}
FULL	tapleaf `0xC2` (BIP-441 restored ops, 64-bit math), `OP_TX`, CSFS (BIP-348), EC ops	everything	1,020 vB
MID	`0xC0`: `OP_CAT`, `OP_TX`, CSFS, `OP_TWEAKADD` (BIP-449)	everything; rewrite $h \leq 5$	1,355 vB
MIN	`0xC0`: `OP_CAT`, `OP_TWEAKADD`, nothing else	exit and **SPLIT**	1,473 vB
CCV	`OP_CHECKCONTRACTVERIFY` (BIP-443) + `OP_TX` (+ CSFS for vouchers), either numerics	everything	908 vB
floor	CCV + a hypothetical `OP_MERKLESUMUPDATE`	measurement only	about 541 vB

- Same tree format in every tier. A rule change lands in every tier that can carry it, or the spec says why not.
- MID's 4-byte `CScriptNum`: leaves arrive as fields, amounts as numbers plus `PADZ` pads.
- Liquid has most of MID today: the obvious place for a prototype.

MIN: introspection recovered from OP_CHECKSIG

MIN has no introspection opcode, but OP_CHECKSIG *does* see the transaction.

1. The script builds the BIP-341 signature message itself from the values it wants to constrain (outputs, amounts) with OP_CAT.
2. It hashes it and computes the signature for key G and nonce G: $s = 1 + e$, a value the script can derive (OP_TWEAKADD).
3. OP_CHECKSIG hashes the *real* transaction. It passes only if the constructed message equals the real one.

What it shows

The signature authorises nothing; it **binds**. Every field the script pinned is enforced.

The load-bearing operation, a one-transaction exit, needs only OP_CAT and one point-tweak opcode.

It's an upper bound on what's required, not a recommendation: 43% more than FULL, and only exit and **SPLIT**.

The floor: one Merkle-sum opcode

Unrolling the fold is most of the script. A single opcode that verifies a Merkle-sum path and returns the updated root replaces the fold and the sum arithmetic along the path. Then:

- the exit script is **252 bytes at any depth**, dense or sparse,
- the proof is one 40-byte sibling per *occupied* level,
- a key-addressed tree of depth 64 is practical,
- an exit is **about 381 + 10**$\log_2 N$ **vB**.

The scripts still do arithmetic for pending receives and lock-time checks.

Status

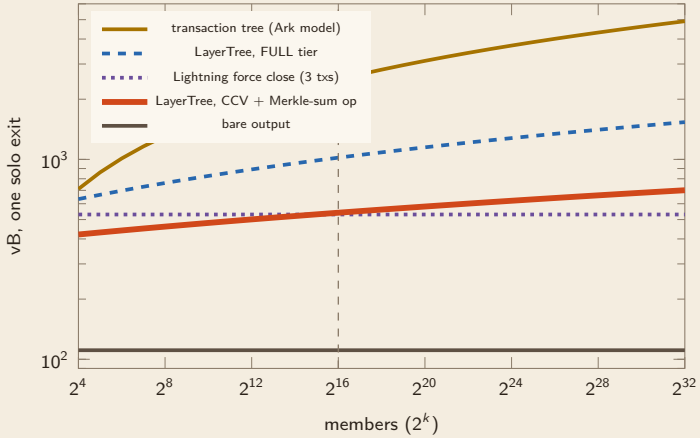
Specified and measured in the simulator behind a `hypothetical` flag (133 checks). Quarantined from every tier and golden vector. Nobody has proposed it for Bitcoin.

Later slides mark the figures that use it (541, 626, 547 vB). The rest are FULL or the named tier.

626 and 547 also assume a proof that can start above height 0; without that, about 480 vB more per merge or rewrite.

Performance and scalability

Leaving alone: 10 vB per doubling

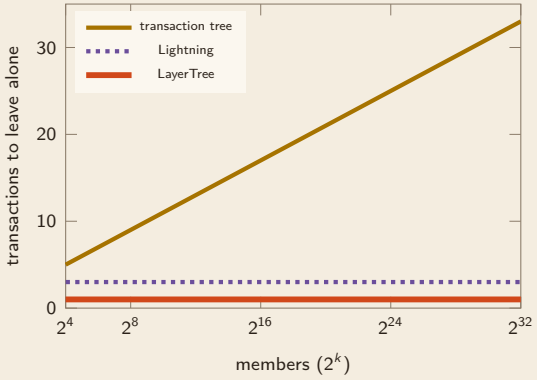


members	LT	FULL	tree
2^8	461	762	1,311
2^{16}	541	1,020	2,511
2^{20}	581	1,149	3,111
2^{32}	701	1,536	4,911

LT is CCV with the hypothetical opcode: $381 + 10 \log_2 N$ vB, one 40-byte sibling per occupied level. A transaction tree pays a whole ∼150 vB transaction per level.

At 2^{16}: 4.6× smaller; at 2^{32}: 7×.

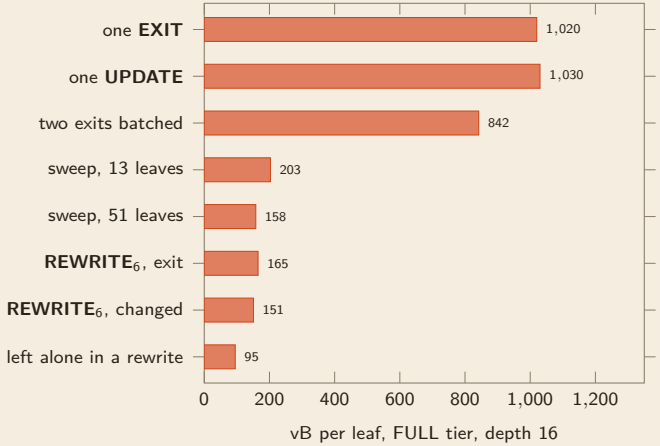
One transaction is the safety property



at 2^{16}	LayerTree	LN close	tx tree
transactions	1	3	17
blocks to spend	1	∼145	16 + leaf lock
who must be online	nobody	a tower	you, before expiry
what expires	nothing	nothing	the round

Each extra transaction is another fee at that day's rate, and another place a low-fee ancestor or a conflicting spend can hold you up. Under fee pressure, or in a mass exit, or with a hostile operator, **one transaction beats seventeen**.

Batching: the per-leaf cost falls with the batch



Owners exiting together each carry a full path, so a pair saves only the fixed overhead.

A cohort shares one path above its subtree: the sweep (operator, after a lease) and the rewrite (owners, any time) hash the interior once.

Tapscript has no loops: the per-position step is published in every batch. That script is most of a batch's cost, and the reason the next ideas focus on batches.

Per member, over a lifetime

vB per member, entering in a batch	never exits alone	exits in a batch	exits alone
LayerTree, CCV + Merkle-sum op	**3**	**161**	**544**
LayerTree, FULL tier	3	161	1,023
Lightning channel	197	197	573
Transaction tree (Ark model)	86	86	2,554
Bare output	154	154	154

- Entry is nearly free: a cohort graft of 1,024 members is 614 vB, **under 1 vB each**, when the operator funds it and members buy balances off-chain.
- A member who's swept at lease end, or sells their balance by voucher, pays almost nothing.
- A member who exits alone pays for the proof, and loses to a bare UTXO and roughly ties Lightning.
- The UTXO set: a tree of any size is **one output**.

When everyone leaves at once

members	one tree	sharded (2^{10})	sharded, cohort exits	tx tree	Lightning
1,024	41	41	11	1	1
65,536	2,622	41	11	18	35
1,048,576	41,944	610	173	274	556

Blocks until every member has left, nobody cooperating; 1 MvB per block; 25 exits per tree per block; cohort exits remove 64 members per transaction at about 165 vB each.

- Every exit spends the same output, so one tree's exits queue: about 25 per block under the descendant limit, up to 64 under cluster mempool.
- One tree of 65,536 drained one stranger at a time takes about 18 days. So: **size each tree by exit throughput**, shard deployments, and **DETACH** a cohort on demand.
- Sharded with cohort exits, LayerTree drains faster than any design in the table. At a million members everyone is block-space bound and the cheaper exit wins.

Trade-offs

Where it loses

- **Against a bare output, on every solo exit.** 541 vB with the hypothetical opcode (1,020 in FULL) against 111. The proof is the price of sharing, and only batching gets it back.
- **Level with a Lightning force close in bytes only with the hypothetical opcode** (541 against 530 vB; FULL is about twice), but one transaction and one block against three and about a day.
- **A non-cooperative mass exit from one big tree.** Exits serialise through one output; transaction-tree branches unroll in parallel.
- **On-chain payments.** An **UPDATE** is about 1,030 vB. Value has to move off-chain: channel leaves, vouchers, or batched rewrites.
- **It needs opcodes that aren't active.** With today's proposals the cheapest exit is CCV's 908 vB; the 541 vB figures assume an opcode nobody has proposed.
- **Joining needs the operator.** No liveness guarantee for getting in, only for getting out.

One output, one queue

Every transition spends and recreates the same UTXO, so unconfirmed transitions form a chain.

- An honest exit can be displaced by any conflicting spend that confirms first, and must be rebuilt against the new root.
- The spec doesn't pretend to derive liveness: assumption **[C]** names it. At most β conflicting spends ahead, κ per block, so an exit lands within $\Delta(q) = \Delta_L(\lceil(\beta + q)/\kappa\rceil + 1)$.
- Fee pinning is outside [C]: one cheap transition early in the queue caps the fee rate of everything behind it.

The structural answers

- shard: many trees, each sized by exit throughput
- **DETACH**$_h$: a cohort gets its own output and its own chain
- batches count once against the limit
- **EXIT**^t_K: a tower rebuilds and rebroadcasts with the owner offline
- cluster mempool raises κ for a cooperative tree

This is the design's real weak spot.

The trust model

The operator can

- add members (graft into empty slots)
- co-sign cooperative changes
- top up a leaf (amount only)
- evict a leaf after its lease, paying it in full
- reclaim a pending leaf after its deadline
- refuse to let you in, and grief cheaply

The operator cannot

- take a satoshi of anyone's leaf
- stop you leaving, or delay you past [C]
- change your keys or shorten your lease
- replay a voucher, or void one you hold in a pending pair before its deadline

What members must do: keep their own path data (proofs are data availability), check their path inside a cohort *before* funding a graft (a fake subtree freezes value, it can't create it), and watch leases and deadlines, or hand that to a tower.

How we know it's right

Rules proved, scripts executed, two implementations agreeing

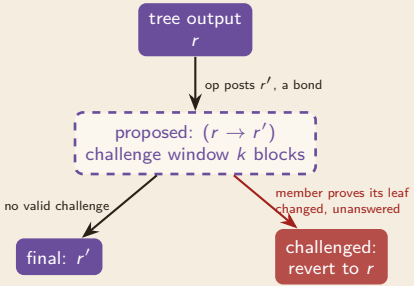
Layer	Checked by
the rules (spec 7)	Lean 4, **72 theorems**, no Mathlib, no `sorry`, axiom-audited
the scripts	a tapscript simulator with the real BIP-341 sighash: 1,610 FULL, 1,391 MID, 57 MIN, 4,295 CCV, 220 channel-leaf checks, positive and negative per rule
Python against Go	byte-exact golden scripts and a shared sighash vector
Go-built witnesses	427 end-to-end transactions from the Go builders, replayed through the simulator
composition	a UC argument with the Lean theorems as extraction lemmas (the paper)

- Conservation holds for every valid sequence of steps, **unconditionally**. Payability (any holder can build an accepted exit) needs the real-tree invariant, which grafting must establish off-chain: the proofs forced that distinction into the open.
- The proofs changed the design: the merge claim's target needs `exit_key` equality, or a cycle of merges and a different-key target were possible.
- Not covered: the link from rules to bytecode is execution, not proof. A P model of the off-chain protocol is designed, not built.

Future directions

Optimistic batches: publish the root, not the leaves

A sketch for discussion: not in the spec, not reviewed.



Today a rewrite publishes every slot twice and verifies everything: about 151 vB per changed leaf.

Optimistically, the operator publishes only the new root (and a commitment to the change set). For k blocks:

- a member proves its leaf under r and its slot under r' differ, or asks for its new path;
- the operator answers with the member's signature over the change, or the path, on-chain;
- no answer: the proposal reverts and the bond pays the challenger.

Happy path: a constant-size transaction, whatever the batch.

What the optimistic version trades

Gains

- batch cost independent of batch size: per-leaf cost toward zero
- fewer bytes than even a native Merkle-sum opcode's batch
- the existing exit stays the escape hatch: nobody's exit gets worse

Costs and open questions

- liveness: someone must watch every window (towers)
- data availability: withheld paths become on-chain challenges
- latency: a batch is final only after k blocks, and the output carries two roots meanwhile
- griefing: bond sizing for both sides
- exits in the window must hold under both roots
- needs CSFS and two-root state (CCV fits)

Today's design is the pessimistic end: every batch carries its own proof, so no window and no watchers.

Other directions

- **Cheaper rewrite steps.** Just landed: an 8-byte amount-only element in place of a republished 116-byte leaf (587 $\rightarrow$ 547 vB per settled pair, with the hypothetical opcode). The per-position script is now most of a batch's cost.
- **A native Merkle-sum opcode.** 252 bytes of script at any depth; key-addressed trees of depth 64. It helps exits, not batches. Worth proposing?
- **Liquid prototype of MID.** Most of the opcodes exist there today.
- **Signet / Inquisition harness** to confirm the throughput and TRUC-anchor assumptions on a real node.
- **The P model** of the off-chain protocol: covenant oracle, watchtower with lag, adversarial timing.
- **A verified compiler** from rules to tapscript, closing the gap the simulator covers today.
- **PTLCs and second-stage HTLCs** for channel leaves.
- **A tower protocol**: registration, payment, hand-off of authorisations and vouchers.
- **Cross-tree swaps** and splicing between shards.

Questions

